CS 635, Fall 2025

Module 6 Assignment Project 3

Prepared by:
Leo Paredes (820558278), Brandon Reynolds (821924175)
E-mail: lparedes7353@sdsu.edu , bareynolds@sdsu.edu
San Diego State University

Table of Contents

1. Introduction..... 3
2. System Architecture Overview.....3
3. Tokenizer (Lexical Analysis)..... 4
4. Abstract Syntax Tree (AST) Design.....5
5. Parsing and Program Construction.....6
6. Interpreter Implementation.....7
7. Geometry and Location Computation..... 8
8. Visitor Pattern Integration..... 8
8.1 Memento Visitor.....9
8.2 Total Distance Visitor..... 9
9. Mock Turtle Class..... 9
10. Framework Integration.....10
11. Coupling and Cohesion Evaluation..... 11
12. Testing Strategy (Using Mock Turtle and Pytest)..... 11
13. Results.....13
14. Graphical User Interface (GUI)..... 14
15. Conclusion..... 14

1. Introduction

This project focuses on the design and implementation of an interpreter for a simplified turtle graphics language. The goal is to build a complete system that can read user programs, translate them into an internal representation, and execute them through a virtual turtle that moves and draws on a canvas. The assignment highlights key ideas in programming language design, including interpretation, the use of abstract syntax trees, and the visitor pattern. These concepts are used together to create a system that is both extensible and easy to understand.

A major aim of this project is to practice good software design through careful attention to coupling and cohesion. Each part of the interpreter is separated into modules that have a clear purpose and minimal dependence on other parts of the program. This encourages a design that is easier to test, reuse, and modify. The project also requires the integration of external frameworks, such as the Python turtle library, to demonstrate how an interpreter can interact with existing tools.

To support testing and verification, a mock turtle is used to simulate the behavior of the real turtle without producing graphical output. This allows the interpreter and visitors to be tested in a predictable environment while still matching the behavior of the real graphics engine. The use of visitors provides additional insight into the program by enabling step by step execution and measurement of turtle motion.

Overall, this project provides practical experience in programming language interpretation, object oriented design, and framework integration when building software systems.

2. System Architecture Overview

The system is organized as a collection of modules that work together to process and execute programs written in the simplified turtle graphics language. The overall workflow begins with a text file and ends with visible graphical output and computed results. Each component has a focused responsibility which reduces unnecessary interaction between modules.

The first stage of the system is the lexer. Its only task is to convert the text file into a stream of tokens that are easier for the parser to work with. The parser then reads these tokens and constructs an abstract syntax tree. This tree represents the structure of the user program in a form that is easier to interpret and analyze. Each command such as forward, left, right, or repeat is represented as a distinct node in the tree.

Execution of the program is handled by the interpreter. The interpreter walks through the abstract syntax tree and issues movement commands to a turtle like object. The turtle object can be a real graphics turtle or a mock turtle that stores state in memory for testing. This separation allows the interpreter to remain independent of the graphical framework and supports both visual output and unit testing.

The visitor subsystem provides a second layer of functionality. Instead of executing the program, visitors perform analysis on the abstract syntax tree. The memento visitor steps through the program and records each state of the turtle. The distance visitor calculates the total distance traveled. Visitors do not modify the program and they do not depend on the graphics engine. This design keeps them reusable and easy to understand.

The framework adapter module provides integration with the Python turtle library. It allows the interpreter to control a real graphical turtle without changing its core logic. This maintains low coupling and supports extensibility if different graphics frameworks are used in the future.

Together these components form a complete system with a clear flow from input to output. The structure highlights the value of modular design since each part has a specific role and can be tested or maintained without affecting the others.

3. Tokenizer (Lexical Analysis)

The lexical analysis stage is the first step in processing a turtle graphics program. The goal of this stage is to transform the raw source text into a sequence of meaningful tokens. These tokens are the basic units that the parser will later interpret. The tokenizer removes concerns about spacing, line breaks, and other formatting details so that the rest of the system can operate on clean and predictable input.

The tokenizer used in this project is intentionally simple because the instruction set language is small. It scans the input character by character and groups them into words, numbers, and bracket symbols. Commands such as forward or left are treated as individual tokens. Numbers that represent distances or angles are captured as separate tokens. Brackets are important for the repeat construct, so they are identified as tokens as well.

This design keeps the tokenizer focused on a single responsibility and prevents unnecessary coupling with the parser or interpreter. The tokenizer does not attempt to understand the meaning of the tokens. It only prepares structured data for later stages of the system. This approach supports cohesion because the module is concerned with one task and performs it in a clear and consistent way.

By keeping the tokenizer small and predictable, the rest of the interpreter can rely on a stable input structure. This simplifies debugging and improves overall clarity. The output of the tokenizer becomes the foundation for parsing and program execution in the later stages of the interpreter.

4. Abstract Syntax Tree (AST) Design

The abstract syntax tree is the internal representation of a turtle graphics program. Its purpose is to capture the structure of the program in a form that is easy for

both the interpreter and the visitors to work with. Instead of operating directly on the raw text or the token stream, the system uses a tree of node objects where each node represents a single command or control structure.

Each command in the language is represented by its own class. For example, forward, left, right, pen up, and pen down each correspond to a distinct node type. These nodes store only the data needed to describe the command, such as the movement distance or turning angle. This keeps each class focused and supports strong cohesion inside the module.

The repeat construct is modeled as a node that contains both the number of repetitions and a list of child nodes that make up its body. This allows the tree to represent nested commands and to capture the hierarchical nature of the language. The abstract syntax tree mirrors the logical structure of the program rather than the format of the text file.

Each node also provides an accept method, which allows visitors to interact with the tree without placing visitor logic inside the nodes themselves. This method supports the visitor pattern and helps maintain low coupling between the tree structure and the analysis tools that operate on it.

The abstract syntax tree design makes it possible to interpret the program, walk through it step by step, compute distances, and integrate new features without changing the core representation.

```
REPEAT 4 [ FORWARD 100 RIGHT 90 ]
LEFT 45
FORWARD 141
PENUP
FORWARD 10
PENDOWN
```

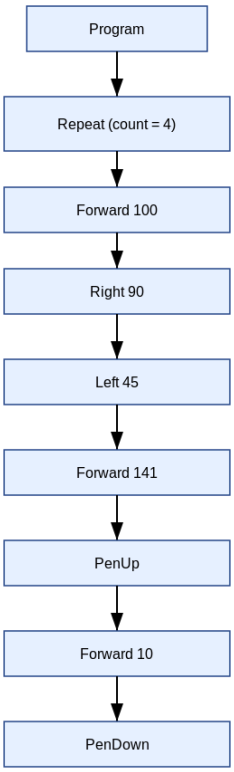

Fig. 1. AST Flowchart for commands

5. Parsing and Program Construction

The parsing stage is responsible for converting the stream of tokens into a structured abstract syntax tree. This stage reads each token in order and decides which type of node should be created based on the command it represents. Its main concerns are identifying commands, reading any required numeric values, and properly handling the repeat construct.

During parsing, the parser walks through the token list and processes one statement at a time. Commands such as forward or left require a numeric value, so the parser reads the next token and converts it into a number. Commands like pen up or pen down do not require additional information and can be created immediately. The repeat command is more involved because it introduces a block of nested statements. When the parser encounters “repeat”, it reads the repetition count, confirms that a bracket begins the block, and then continues parsing statements until the closing bracket appears.

The parsing logic is designed to be clear and easy to follow. Each part of the parser focuses on one specific rule of the language, which helps maintain good cohesion within the module. The parser does not execute commands or perform analysis. Its only role is to build the abstract syntax tree that will later be used by other components of the system. This separation reduces coupling and supports testing because the tree can be examined independently.

By the end of parsing, the system has a complete representation of the program in the form of a list of abstract syntax tree nodes. This representation captures the logical flow of the user program and provides the structure needed for interpretation, step analysis, and measurement of turtle movement.

6. Interpreter Implementation

The interpreter is the component that executes the abstract syntax tree and produces the behavior described by the user program. Its task is to walk through the tree and call the appropriate methods on a turtle like object. The interpreter does not know whether the turtle is a real graphics turtle or a mock turtle. It only relies on the shared interface that provides forward, left, right, pen up, and pen down operations. This design keeps the interpreter independent from the graphics framework and improves the flexibility of the system.

The interpreter processes the program by visiting each node in order. For a forward node, it instructs the turtle to move by the given distance. For left and right nodes, it changes the turtle heading accordingly. Pen commands modify the drawing state of the turtle. The repeat node is handled by executing its body the specified number of times. The interpreter is simple by design because each node type corresponds to one clear action.

This module shows strong cohesion because it focuses on a single purpose.. It only executes commands based on the structure already created by the parser. The interpreter also avoids unnecessary coupling by depending only on the turtle interface rather than any specific implementation. As a result, it can work with the mock turtle for testing or with the real turtle adapter for graphical output without any changes to its logic. It is the connector between the abstract structure of the program and the concrete actions taken by the turtle.

7. Geometry and Location Computation

The geometry and location computation component is responsible for updating the position and orientation of the turtle as it moves through the program. These calculations are performed inside the turtle implementation rather than in the interpreter. This design keeps mathematical concerns separate from program control logic and helps maintain low coupling between modules.

The turtle stores its current position as x and y coordinates, and its current direction as a heading measured in degrees. When the interpreter issues a forward command, the turtle converts the heading into radians and uses basic trigonometric functions to compute the change in position. The new x coordinate is determined by the cosine of the heading multiplied by the movement distance. The new y coordinate is determined by the sine of the heading multiplied by the same distance.

Turning commands modify the heading without changing location. The left command increases the heading by a specified angle while the right command decreases it. Pen state changes do not affect geometry but are recorded so that drawing behavior can be preserved by the graphics framework.

These computations are identical for both the mock turtle and the real turtle adapter. The mock turtle performs the math directly and stores the resulting state in memory. The real turtle adapter updates its state by calling the underlying Python turtle library. By keeping the geometric logic consistent across both implementations, the system ensures that the interpreter and the visitors behave predictably in testing and in graphical execution.

The geometry module is cohesive because it focuses only on the movement rules of the turtle. It does not interpret commands or analyze the program. This separation creates a clear boundary between program logic and mathematical computation.

8. Visitor Pattern Integration

The visitor pattern is used in this project to separate program analysis from program execution. Instead of placing analysis logic inside the interpreter or the abstract syntax tree, visitors provide a flexible way to walk through the tree and perform additional tasks. This pattern keeps the core system clean and allows new behaviors to be added without changing existing components. The accept method on each node allows a visitor to operate on the tree structure in a uniform and organized way.

Visitors in this project do not draw graphics or modify the real turtle. They work with a mock turtle so that computations remain predictable and easy to test. This separation supports low coupling and makes it possible to reuse visitors in any environment.

8.1 Memento Visitor

The memento visitor provides step by step tracking of the turtle state as the program executes. Each time the visitor encounters a command, it applies the command to the mock turtle and then records a snapshot of the turtle position, heading,

and pen state. These snapshots are stored as memento objects that can be reviewed later.

This visitor allows the program to be replayed or examined at any point in its execution. It supports debugging and demonstration because it reveals how each command affects the turtle. The memento visitor does not change the structure of the program and does not require any changes to the interpreter. It only depends on the abstract syntax tree and the simple interface of the mock turtle.

By collecting states rather than drawing graphics the memento visitor remains focused on program analysis.

8.2 Total Distance Visitor

The total distance visitor is designed to calculate the total distance traveled by the turtle while executing a program. It walks through the abstract syntax tree and adds the distance value for each forward command it encounters. Repeat blocks are handled by visiting their children the correct number of times.

This visitor focuses only on the amount of movement, not the direction or the final position. Even if the turtle returns to its starting point, the visitor still reports the full traveled distance. This gives a measure of the total motion contained in the program. Because it does not depend on graphics or state snapshots, the total distance visitor remains simple and cohesive.

9. Mock Turtle Class

The mock turtle class serves as a lightweight substitute for the real graphics turtle and is an essential part of the testing and analysis process in this project. Its purpose is to replicate the behavior of the real turtle without producing any visible drawing. This allows the interpreter and the visitors to run in a predictable environment where position, heading, and pen state can be examined directly.

The mock turtle stores its current coordinates, heading, and pen status in simple data fields. It updates these values using the same geometric rules as the real turtle adapter. Forward commands adjust the position based on trigonometric calculations, and left and right commands modify the heading. Pen state changes are recorded but do not affect movement. Because the mock turtle does not interact with a graphics window, it avoids side effects that can make testing more difficult.

This class is especially important for the visitor pattern. Both the memento visitor and the total distance visitor rely on the mock turtle to compute results without altering the real graphical output. The mock turtle gives these visitors a stable and reproducible way to interpret the program and gather information. It also allows unit tests to compare expected and actual states with simple numerical checks.

The mock turtle supports low coupling because it shares the same public interface as the real turtle adapter. The interpreter does not need to know which implementation it is using. This interchangeable design improves flexibility and makes the system easier to extend.

Overall, the mock turtle plays a central role in achieving accurate testing and clean separation between graphical output and internal analysis. It strengthens the structure of the interpreter by allowing verification of logic without relying on graphical effects or external frameworks.

10. Framework Integration

Framework integration in this project focuses on connecting the interpreter to the Python turtle module through an adapter class. The goal is to allow the interpreter to drive an actual graphical turtle without creating direct dependencies on the graphics framework. This approach supports low coupling and keeps the interpreter flexible and reusable.

The system introduces a real turtle adapter that wraps the Python turtle object and exposes the same interface as the mock turtle. The adapter forwards calls such as forward, left, right, pen up, and pen down to the underlying graphics engine. It also collects position and heading information by reading these values from the real turtle when needed. Because the adapter presents a consistent interface, the interpreter can switch between the mock turtle and the real turtle without any changes to its logic.

The adapter also plays an important role in extensibility because it allows the interpreter to work with different graphics libraries without changing the core modules. If a new graphics framework were introduced in the future, a matching adapter could be created while leaving the interpreter, parser, and abstract syntax tree untouched. By isolating all framework specific logic inside the adapter, the project shows how external libraries can be integrated in a clean and organized way. This separation improves maintainability and supports a more modular approach to building interpreters and graphical tools.

11. Coupling and Cohesion Evaluation

The design of this interpreter places a strong emphasis on achieving low coupling and high cohesion, which are central goals in effective software architecture. Each module in the system is responsible for one clear task, and modules interact only through simple and well defined interfaces. This structure improves readability, supports testing, and makes the system easier to extend.

Cohesion is maintained by grouping related functionality into focused modules. The lexer is concerned only with token generation. The parser is responsible only for

constructing the abstract syntax tree. The interpreter handles execution but does not perform parsing or analysis. The mock turtle and real turtle adapter manage geometric state updates and graphical behavior. Visitors perform analysis without contributing to execution. By keeping responsibilities tightly grouped, each module remains understandable and easier to maintain.

Low coupling is achieved through the use of shared interfaces and clear abstraction boundaries. The interpreter does not depend on any specific turtle implementation. It works with both the mock turtle and the real turtle adapter because they present the same set of operations. Visitors depend only on the abstract syntax tree and the turtle interface, and not on the interpreter or graphical frameworks. This separation prevents changes in one module from forcing changes in others, which reduces the risk of introducing errors and improves system stability.

The use of the adapter pattern further strengthens low coupling by isolating framework specific logic in a single component. This allows the interpreter to remain independent of the Python turtle module. New frameworks or graphical tools can be integrated without altering the rest of the system.

12. Testing Strategy (Using Mock Turtle and Pytest)

The testing strategy for this project focuses on verifying the correctness of the interpreter, the abstract syntax tree, and the visitor pattern without relying on graphical output. The Python turtle module produces visual results that are difficult to test automatically, so the project uses a mock turtle to create a controlled and reproducible environment. This allows the test suite to evaluate program behavior in a precise way.

The mock turtle replicates the movement rules, heading changes, and pen state updates of the real turtle while storing all information in memory. Because it does not draw on a screen, its state can be inspected directly in unit tests. This makes it possible to check positions, angles, and recorded states with simple numerical comparisons. The mock turtle therefore plays an important role in separating the logic of the interpreter from the concerns of graphics rendering.

Pytest is used as the testing framework due to its simplicity and clear output. The test suite includes separate tests for tokenization, parsing, interpretation, geometry, and visitor behavior. Lexer tests verify that input programs are converted correctly into token sequences. Parser tests confirm that the abstract syntax tree is built with the correct structure. Interpreter and geometry tests use the mock turtle to confirm that movement and orientation calculations are accurate. Visitor tests check that mementos are recorded in the expected order and that the total distance visitor measures the full traveled distance.

This structured testing approach supports strong cohesion within modules and low coupling across components. Each test verifies a small and focused aspect of the

system, which helps locate issues quickly and ensures the reliability of the full interpreter pipeline. By avoiding dependency on graphical output, the tests remain consistent across platforms and environments.

13. Results

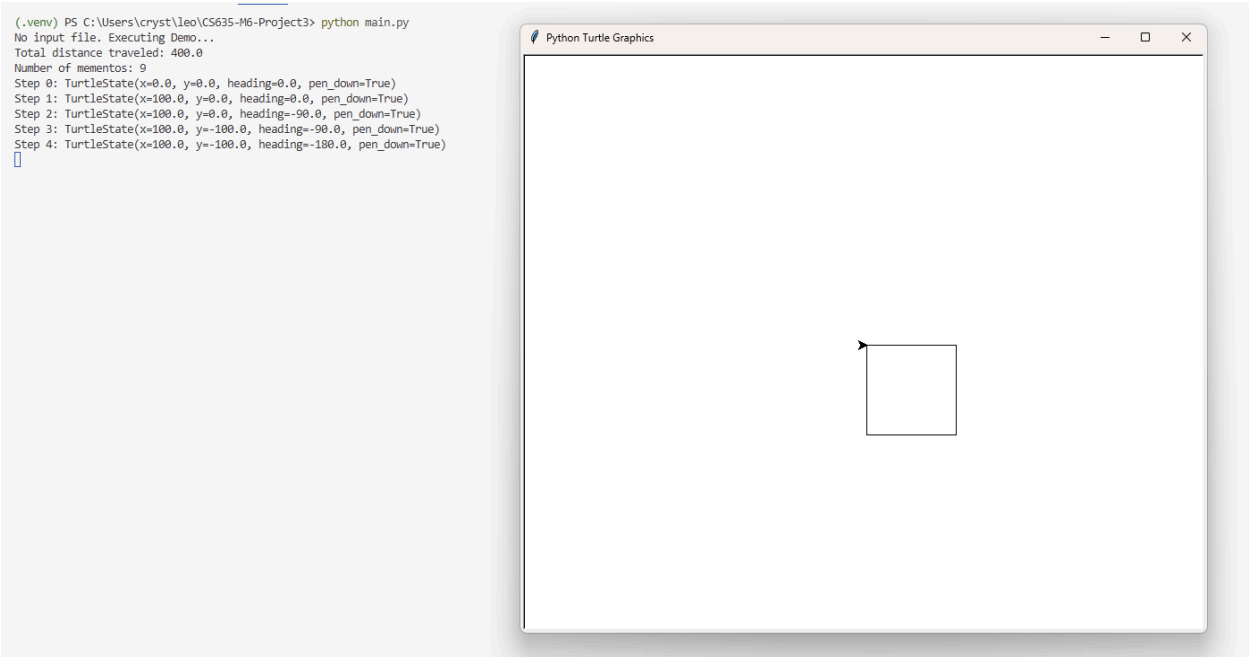

Fig. 2. Running ‘python main.py’

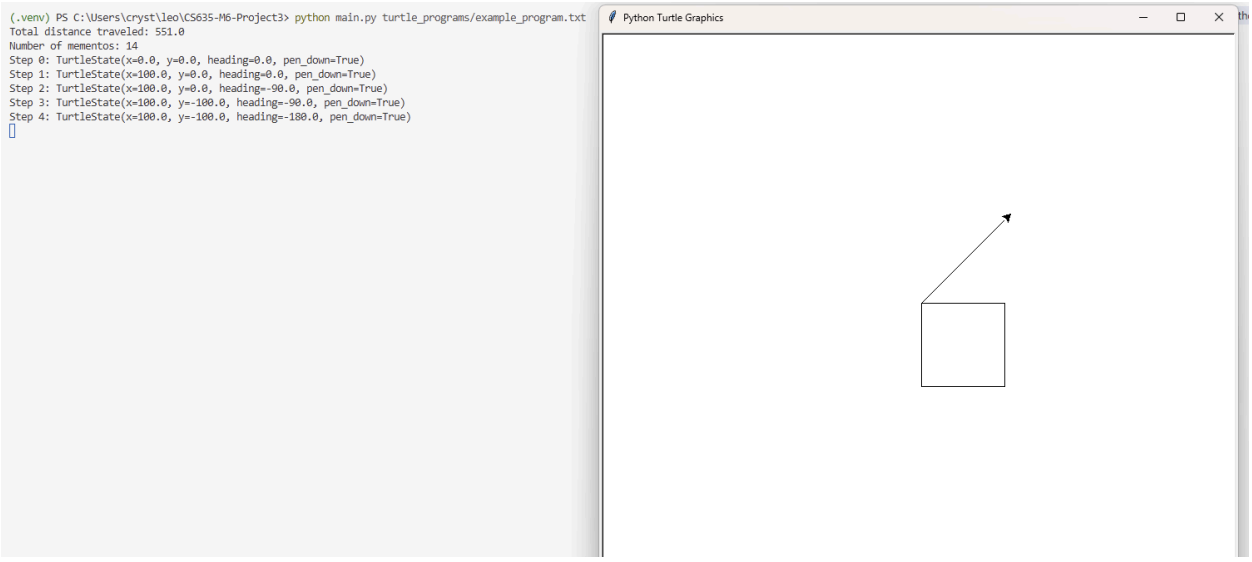

Fig. 3. Running ‘python main.py turtle_programs/example_program.txt’

```
(.venv) PS C:\Users\cryst\leo\CS635-M6-Project3> pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.13.9, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\cryst\leo\CS635-M6-Project3\.venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.13.9', 'Platform': 'Windows-11-10.0.26100-SP0', 'Packages': {'pytest': '9.0.2', 'pluggy': '1.6.0'}, 'Plugins': {'html': '4.1.1', 'metadata': '3.1.1'}}
rootdir: C:\Users\cryst\leo\CS635-M6-Project3
configfile: pytest.ini
testpaths: tests
plugins: html-4.1.1, metadata-3.1.1
collected 8 items

tests/test_interpreter_and_geometry.py::test_square_returns_near_start_and_heading_zero PASSED [ 12%]
tests/test_interpreter_and_geometry.py::test_forward_and_turn_geometry PASSED [ 25%]
tests/test_lexer.py::test_tokenize_simple_program PASSED [ 37%]
tests/test_lexer.py::test_tokenize_with_brackets_and_newlines PASSED [ 50%]
tests/test_parser.py::test_parse_single_forward PASSED [ 62%]
tests/test_parser.py::test_parse_repeat_block PASSED [ 75%]
tests/test_visitors.py::test_distance_visitor_square PASSED [ 87%]
tests/test_visitors.py::test_memento_visitor_steps PASSED [100%]

============================== 8 passed in 0.02s ==============================
(.venv) PS C:\Users\cryst\leo\CS635-M6-Project3>
```

Fig. 4. Pytest Results

14. Graphical User Interface (GUI)

Fig. 5. Graphical User Interface (GUI) with commands

In order to make the project more user friendly we sought to use a Graphical User Interface (GUI) that could load commands and visualize the steps. When the GUI is launched, the left side holds the code editor similar to an Integrated development environment (IDE) which will highlight typed words such as FORWARD, REPEAT, LEFT, RIGHT, and numbers which provides the basic syntax for our language.

To get started users can start developing through either loading a .txt file through “Load File” or using the text editor. Once a file is loaded or commands have been inputted, the user can click “Run” in order to see their commands in action. The editor’s text is then tokenized and then parsed to form an Abstract Syntax Tree (AST) from the

commands. Then the interpreter then executes each AST node sequentially with a slight delay to visualize the turtle moving each step.

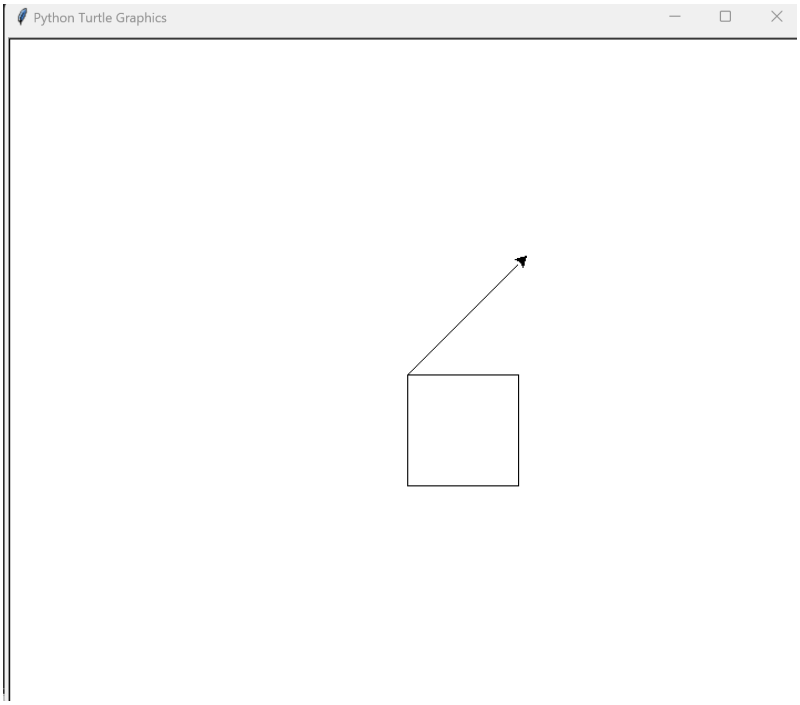

Fig. 6. GUI showing visual when “Run” is pressed

If a user would like to step through the commands incrementally then the “Step” option is available as well. The button applies updated states to the turtle which allows the turtle's movement to be viewed step by step.

Meanwhile the bottom right side of the GUI canvas displays the computed total distance the turtle travels. It includes the total distance, current step, and number of mementos in order to give real time feedback about the program execution.

Lastly the right side of the GUI supports zoom and pan along with the ability to export drawings. Users can choose between exporting the canvas created as a png.

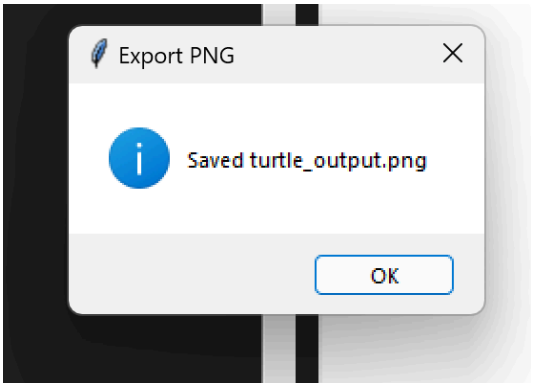

Fig. 7. Popup for when .png is being saved

15. Conclusion

This project brought together several core ideas in programming language design and object oriented software development. The interpreter for the simplified turtle graphics language demonstrated how source code can be processed through a sequence of structured stages, beginning with tokenization, parsing, abstract syntax tree construction, and execution. Each component of the system contributed to a modular workflow that reinforced the value of good software architecture design.

The use of the visitor pattern provided a flexible method for analyzing programs without altering the interpreter or the abstract syntax tree. The memento visitor and the total distance visitor each offered a different perspective on program execution and illustrated how new behaviors can be introduced without modifying existing components. The geometry model and the mock turtle class made it possible to test the logic of the system in a controlled environment.

Throughout the project, attention to coupling and cohesion guided the organization of modules and the design of interfaces. The adapter for the Python turtle framework showed how external tools can be integrated cleanly while keeping the core system independent and reusable. These design choices helped create a system that is easy to test, extend, and maintain.

Overall, the project provided practical experience with interpretation, abstract syntax trees, object oriented patterns, and framework integration. It connected theoretical concepts with a hands-on approach and demonstrated how a good design can support both clarity and flexibility in software systems.